watchEffect

watchEffect 立即执行，没有immediate

watchEffect 不需要传递侦听的内容，自动会感知代码的依赖，不需要传很多参数，只需要传第一个回调函数，不能获取之前的值

如果说需要停用侦听器，可以回调这个侦听器函数

```
<template>
   <h1>这是Home组件</h1>
   <h4>num: {{num}}</h4>
   <button @click="stop">停用侦听器</button>
</template>


import {ref,watchEffect,toRefs,watch} from 'vue'
   export default{
      setup(props,context){ //composition API 的入口
         const num = ref(0)
         const str = ref('测试')
         /*
            watchEffect 立即执行，没有immediate
            watchEffect 不需要传递侦听的内容，自动会感知代码的依赖，不需要传很多参
数，
            只需要传第一个回调函数，不能获取之前的值
          */
        const stop=  watchEffect(()=>{
           //如果说需要停用侦听器
            //以回调这个侦听器函数
            console.log('num:',num.value)//ref必须通过属性value拿到值
            console.log('str:',str.value)//ref必须通过属性value拿到值
         })
         setInterval(()=>{
           num.value++;
           },1000)
         // 要在template模板上使用的变量，必须通过return，返回出去，才能在template
模板上使用
         return{
            num,
            stop
         }
      }
   }
```

watch

```
<template>
    <h4>num: {{num}}</h4>
    <h4>id: {{id}}</h4>
    <input type="text" v-model="uname">
    <input type="text" v-model="id">
    <button @click="stop">停用侦听器</button>
</template>
//同上，要用到的方法必须引入，这里就不再书写
//在setup中开启监听
        setup(props,context){ //composition API 的入口
            const num = ref(0)
            const str = ref('测试')
            const state =reactive ({
                id:101,
                uname:'莫尘'
            },)
            // 只要num有变化，会触发watch的回调,watch会在创建时自动调用
            // watch(()=>{console.log(num.value)})
            // 指定了要侦听的值
            //---------------侦听 ref 的数据源----------------------------------------
            watch(num,(val,oval)=>{
                // val:新的值,oval：之前的值
                // console.log("c num:",num.value)
                console.log(val,oval)
            },{//第二个参数obj，immediate、deep
                immediate:true //默认就是只有数据改变时才会监听，第一次不会执行，设置
true第一次就会执行
            })
            //--------------侦听 reactive 数据源----------------------------------------
            //1.1 第一种方式侦听
            // 侦听state下的id属性数据的变化
            watch(state,(val,oval)=>{
                // console.log("id: ",val.id,oval)
            },{
                immediate:true, //默认就是只有数据改变时才会监听，第一次不会执行，设置
true第一次就会执行
                deep:true,//开启深度侦听,能够侦听到对象的属性值的变化
            })
            //1.2 第二种方式侦听
            // 侦听 state.uname属性
            //object.values(toRefs(state))
            watch(()=> state.uname,(uname,p)=>{
                // uname:新值,p:旧的值
                console.log(uname,p)
```

```
        },{
            immediate:true
        })
        //1.3 侦听多个数据源
        // 侦听多个数据(id,uname)
        // ()=>state.id ,//object.values(toRefs(state))
        const stop=watch([()=>state.id,()=>state.uname] ,([id,uname],
[oid,oname])=>{
            // id 新的值，oid旧的值
            console.log("id:",id,oid)
            // uname:新，oname:旧
            console.log("uname:",uname,oname)
        })

        //解构后赋值给常量s
       const s= toRefs(state)
        watch(s.uname,(uname,p)=>{
            console.log(uname,p)// uname:新值,p:旧的值
        },{
            immediate:true
        })
        return{
            num,
            stop,
            ...toRefs(state)
        }
    }
```